EECE 7205: Fundamentals of Computer Engineering
Help Desk Ticket System Mini Project

Nashita Hasan, Katherine Tse NUID: 002750189, 003936627
Email: hasan.nas@northeastern.edu, tse.kat@northeastern.edu

Due: November, 2026

1 Feature 1: Classes

Algorithm 1.1 F1: Initialize Classes

```
1: class Ticket
2:     Public:
3:         ID
4:         Name
5:         Category
6:         Minutes
7:         NextTicket
8:     function Ticket(id, name, cat, min)
9:         ID ← id
10:        Name ← name
11:        Category ← cat
12:        Minutes ← min
13:        NextTicket ← NULL
14:    end function
15: end class
16: class TicketQueue
17:    Private:
18:        Front
19:        Rear
20:        Count
21:        nextID
22:        Actions
23:    Public:
24:    function TicketQueue( )
25:        Front ← NULL
26:        Rear ← NULL
27:        Count ← 0
28:        nextID ← 0
29:        Actions ← ActionStack( )
30:    end function
31: end class
```

Algorithm 1.1 F1: Initialize Classes [continued]

```
32: class Action
33:     Public:
34:       Ticket
35:       Action
36:       NextAction
37:     function ACTION(ticket, action)
38:         Ticket ← ticket
39:         Action ← action
40:         NextAction ← NULL
41:     end function
42: end class
43: class ActionStack
44:     Private:
45:       Top
46:       Count
47:     Public:
48:     function ACTIONSTACK( )
49:         Top ← NULL
50:         Count ← 0
51:     end function
52: end class
```

2 Feature 2: Add and Serve

Algorithm 2.1 F2: Add and Serve Tickets

```
1: class TicketQueue
2:     Public:
3:     function ADD(name, cat, min)
4:         newTicket ← TICKET(nextID, name, cat, min)
5:         nextID ← nextID + 1
6:         if Front = NULL then
7:             Front ← newTicket
8:         else
9:             Rear->NextTicket ← newTicket
10:        end if
11:        Rear ← newTicket
12:        Count ← Count + 1
13:        Print("New Ticket Added! Ticket ID: ", newTicket->ID)
14:        Actions.ADDACTION(newTicket, "Add")
15:    end function
16:    function SERVE( )
17:        if Front = NULL then
18:            return
19:            Print("There are no tickets! The Queue is empty.")
20:        end if
21:        currTicket ← Front
22:        Front ← currTicket->NextTicket
23:        if Front = NULL then
24:            Rear ← NULL
25:        end if
26:        currTicket->NextTicket ← NULL
27:        Count ← Count - 1
28:        Print("Ticket: ", currTicket->ID, " was served in ", currTicket->Minutes, " minutes.
29:          Requestor: ", currTicket->Name, " Category: ", currTicket->Category)
30:        Actions.ADDACTION(currTicket, "Serve")
31:    end function
32: end class
```

3 Feature 3: Print

Algorithm 3.1 F3: Print Tickets

```
1: class TicketQueue
2:     Public:
3:     function PRINT( )
4:         if Front = NULL then
5:             Print("There are no tickets! The Queue is empty.")
6:             return
7:         end if
8:         totalMinutes ← 0
9:         currTicket ← Front
10:        while currTicket ≠ NULL do
11:            Print("Ticket ", currTicket->ID, " - Requestor: ", currTicket->Name,
12:                " Category: ", currTicket->Category, " Task Time: ", currTicket->Minutes)
13:            totalMinutes ← totalMinutes + currTicket->Minutes
14:            currTicket ← currTicket->NextTicket
15:        end while
16:        Print("Total Waiting Minutes: ", totalMinutes)
17:    end function
18: end class
```

4 Feature 4: Search with ID or Name

Algorithm 4.1 F4: Search Tickets with ID or Name

```
1: class TicketQueue
2:     Public:
3:     function SEARCH(id)
4:         if Front = NULL then
5:             Print("There are no tickets! The Queue is empty.")
6:             return
7:         end if
8:         totalMinutes ← 0
9:         position ← 1
10:        currTicket ← Front
11:        while currTicket ≠ NULL do
12:            if currTicket->ID = id then
13:                Print("Ticket ", currTicket->ID)
14:                Print("Ticket position in line: ", position)
15:                Print("Estimated wait time: ", totalMinutes, " minutes.")
16:                return
17:            end if
18:            totalMinutes ← totalMinutes + currTicket->Minutes
19:            currTicket ← currTicket->NextTicket
20:            position ← position + 1
21:        end while
22:        Print("Ticket was not found in the queue.")
23:    end function
24:    function SEARCH(name)
25:        if Front = NULL then
26:            Print("There are no tickets! The Queue is empty.")
27:            return
28:        end if
29:        totalMinutes ← 0
30:        position ← 1
31:        currTicket ← Front
32:        while currTicket ≠ NULL do
33:            if currTicket->Name = name then
34:                Print("Ticket for ", currTicket->Name)
35:                Print("Position in line: ", position)
36:                Print("Estimated wait time: ", totalMinutes, " minutes.")
37:                return
38:            end if
```

Algorithm 4.1 F4: Search Tickets with ID or Name [continued]

```
39:             totalMinutes ← totalMinutes + currTicket->Minutes
40:             currTicket ← currTicket->NextTicket
41:             position ← position + 1
42:         end while
43:         Print("Ticket was not found in the queue.")
44:     end function
45: end class
```

5 Feature 5: Cancel ID

Algorithm 5.1 F5: Cancel Tickets

```
1: class TicketQueue
2:     Public:
3:     function CANCEL(id)
4:         if Front = NULL then
5:             Print("There are no tickets! The Queue is empty.")
6:             return
7:         end if
8:         currTicket ← Front
9:         prevTicket ← NULL
10:        while currTicket ≠ NULL do
11:            if currTicket->ID = id then
12:                if currTicket = Front then
13:                    Front ← currTicket->NextTicket
14:                    if Front = NULL then
15:                        Rear ← NULL
16:                    end if
17:                else if currTicket = Rear then
18:                    prevTicket->NextTicket ← NULL
19:                    Rear ← prevTicket
20:                else
21:                    prevTicket->NextTicket ← currTicket->NextTicket
22:                end if
23:                currTicket->NextTicket ← NULL
24:                Count ← Count - 1
25:                Print("Ticket ", id, " cancelled.")
26:                Actions.ADDACTION(currTicket, "Cancel")
27:                return
28:            end if
29:            prevTicket ← currTicket
30:            currTicket ← currTicket->NextTicket
31:        end while
32:        Print("Ticket was not found in the queue.")
33:    end function
34: end class
```

6 Feature 6: Undo

Algorithm 6.1 F6: Undo Actions

```
1: class ActionStack
2:     Public:
3:     function AddAction(ticket, action)
4:         newAction ← Action(ticket, action)
5:         newAction->NextAction ← Top
6:         Top ← newAction
7:         Count ← Count + 1
8:     end function
9:     function PopAction( )
10:        if Top = NULL then
11:            return NULL
12:        end if
13:        topAction ← Top
14:        Top ← Top->NextAction
15:        topAction->NextAction ← NULL
16:        Count ← Count - 1
17:        return topAction
18:    end function
19: end class
20: class TicketQueue
21:    Public:
22:    function Undo( )
23:        currAction ← Actions.PopAction( )
24:        if currAction = NULL then
25:            Print("Nothing to undo.")
26:            return
27:        end if
28:        if currAction->Action = "Add" then
29:            if Front = currAction->Ticket then
30:                Front ← NULL
31:                Rear ← NULL
32:            else
33:                currTicket ← Front
34:                while currTicket->NextTicket ≠ currAction->Ticket do
35:                    currTicket ← currTicket->NextTicket
36:                end while
37:                Rear ← currTicket
38:                currTicket->NextTicket ← NULL
39:            end if
```

Algorithm 6.1 F6: Undo Actions [continued]

```
40:             Count ← Count - 1
41:             Delete(currAction->Ticket)
42:             Print("Add Undone.")
43:         else if currAction->Action = "Serve" then
44:             currAction->Ticket->NextTicket ← Front
45:             Front ← currAction->Ticket
46:             if Rear = NULL then
47:                 Rear ← currAction->Ticket
48:             end if
49:             Count ← Count + 1
50:             Print("Serve Undone.")
51:         else if currAction->Action = "Cancel" then
52:             if Front = NULL then
53:                 Front ← currAction->Ticket
54:                 Rear ← currAction->Ticket
55:             else
56:                 currTicket ← Front
57:                 prevTicket ← NULL
58:                 while currTicket ≠ NULL AND currTicket->ID < currAction->Ticket->ID
    do
59:                     prevTicket ← currTicket
60:                     currTicket ← currTicket->NextTicket
61:                 end while
62:                 if prevTicket = NULL then
63:                     currAction->Ticket->NextTicket ← Front
64:                     Front ← currAction->Ticket
65:                 else
66:                     currAction->Ticket->NextTicket ← currTicket
67:                     prevTicket->NextTicket ← currAction->Ticket
68:                     if currTicket = NULL then
69:                         Rear ← currAction->Ticket
70:                     end if
71:                 end if
72:             end if
73:             Count ← Count + 1
74:             Print("Cancel Undone.")
75:         else
76:             Print("Invalid Action")
77:         end if
78:         Delete(currAction)
79:     end function
80: end class
```

7 Feature 7: Load and Save File

Algorithm 7.1 F7: Load and Save Tickets into File

```
1: class TicketQueue
2:     Public:
3:     function LOAD(file)
4:         loadFile ← OpenToRead(file)
5:         if loadFile = NOT OPEN then
6:             Print("Could Not Open File.")
7:             return
8:         end if
9:         row[ ]
10:        line, word
11:        while GetLine(loadFile, line) ≠ NULL do
12:            row.Clear()
13:            stringStream ← line
14:            while ParseByComma(stringStream, word) ≠ NULL do
15:                row.Append(Trim(word))
16:            end while
17:            if row.Size() < 3 then
18:                continue
19:            end if
20:            minutes ← StringToInt(row[2]) ADD(row[0], row[1], minutes)
21:        end while
22:        loadFile.Close()
23:        Print("Tickets from file added to queue!")
24:    end function
25:    function SAVE(file)
26:        saveFile ← OpenToWrite(file)
27:        if saveFile = NOT OPEN then
28:            Print("Could Not Open File.")
29:            return
30:        end if
31:        currTicket ← Front
32:        while currTicket ≠ NULL do
33:            saveFile.WriteLine(currTicket->Name, ",", currTicket->Category, ",",
    currTicket->Minutes)
34:            currTicket ← currTicket->NextTicket
35:        end while
36:        saveFile.close()
37:        Print("Saved tickets in queue!")
38:    end function
39: end class
```